SMART FINANCIAL WELLNESS PLATFORM FOR PERSONALIZED EXPENSE MANAGEMENT AND FINANCIAL PLANNING

PHASE I REPORT

Submitted by

AJAY SRINIVAS R 230701017

ADITHYAA SURESH 230701014

ANIRUDH C 230701028

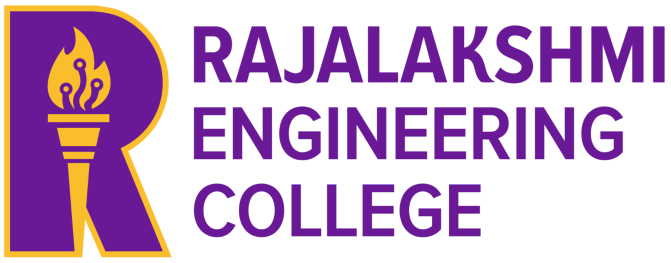
ARVIND RAVI 230701038

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING





RAJALAKSHMI ENGINEERING COLLEGE, CHENNAI

ANNA UNIVERSITY: CHENNAI 600 025

OCTOBER 2025

ANNA UNIVERSITY, CHENNAI

BONAFIDE CERTIFICATE

Certified that this Report titled "**SMART FINANCIAL WELLNESS PLATFORM FOR PERSONALIZED EXPENSE MANAGEMENT AND FINANCIAL PLANNING**" is the bonafide work of **AJAY SRINIVAS R (230701017), ADITHYAA SURESH (230701014), ANIRUDH C (230701028), and ARVIND RAVI (230701038),** who carried out the work under my supervision. Certified further that, to the best of our knowledge, the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidates.

SIGNATURE

Dr. J. Manoranjini,
Professor &
Head of The Department,
Department of Computer Science
and Engineering,
Rajalakshmi Engineering College,
Chennai - 602105.

SIGNATURE

Dr. E. M Malathy,
Supervisor
Professor,
Department of Computer Science
and Engineering,
Rajalakshmi Engineering College,
Chennai - 602105.

Submitted to Project Viva Voce Examination held on _____

INTERNAL EXAMINER **EXTERNAL EXAMINER**

ABSTRACT

Personal financial management systems usually rely on manual expense entry, which can make regular tracking time-consuming and can lead to low user engagement. Automated financial collection can reduce this effort, but it may require sensitive financial data to be processed or stored outside the user's device. Existing approaches may also miss some of the transactions and fail to account for expenses paid in cash. This paper presents Fintor, a smart personal financial management system that automates expense collection while keeping sensitive financial data on the user's device. Fintor combines Android SMS and notification listeners to capture bank alerts and Unified Payments Interface (UPI) transaction notifications, along with on-device optical character recognition (OCR) for extracting information from physical receipts for transactions made using cash. The collected data is processed locally on the user's device to extract the key transaction details before being categorized. A SHA-256-based hashing mechanism is used to detect duplicate transactions received through multiple sources. The transaction records are stored locally using SQLite and synchronized with a MongoDB backend using an idempotent synchronization process when required. The application also analyzes spending patterns to provide budget alerts and personalized financial recommendations. By combining automated expense collection, local processing, receipt extraction, and predictive spending analysis, Fintor aims to reduce manual effort while giving users greater control over their financial data.

ACKNOWLEDGEMENT

Initially we thank the Almighty for being with us through every walk of our life and showering his blessings through the endeavor to put forth this report. Our sincere thanks to our Chairman **Mr. S.MEGANATHAN, B.E, F.I.E.**, our Vice Chairman **Mr. ABHAY SHANKAR MEGANATHAN, B.E., M.S.**, and our respected Chairperson **Dr. (Mrs.) THANGAM MEGANATHAN, Ph.D.**, for providing us with the requisite infrastructure and sincere endeavoring in educating us in their premier institution.

Our sincere thanks to **Dr. S.N. MURUGESAN, M.E., Ph.D.**, our beloved Principal for his kind support and facilities provided to complete our work in time. We express our sincere thanks to **Dr. J. Manoranjini**, Professor and Head of the Department of Computer Science and Engineering for her guidance and encouragement throughout the project work. We convey our sincere and deepest gratitude to our internal guide, **Dr. E.M. Malathy**, Professor, Department of Computer Science and Engineering for her valuable guidance throughout the course of the project. We thank our Project Coordinator, **Mr. Madhusudhanan,** Assistant Professor, Department of Computer Science and Engineering for his useful tips during our review to build our project.

AJAY SRINIVAS : 230701017

ADITHYAA SURESH : 230701014

ANIRUDH C : 230701028

ARVIND RAVI: 230701038

TABLE OF CONTENTS

CHAPTER NO.		TITLE	PAGE NO.
		ABSTRACT	**ii**
		LIST OF TABLES	**vii**
		LIST OF FIGURES	**viii**
		LIST OF ABBREVIATIONS	**ix**
1		**INTRODUCTION**	**1**
	1.1	General	1
	1.2	Objectives	2
	1.3	Existing System	3
	1.4	Proposed System	4
	1.5	Scope of the Project	5
	1.6	Organization of the report	5
2		**LITERATURE SURVEY**	**6**
	2.1	Literature Review	6
	2.2	Comparison and Discussion	10
	2.3	Conclusion	11
3		**PROPOSED SYSTEM**	**13**
	3.1	System Architecture	13
	3.2	System Requirements	14

3.2.1 Software Requirements 14
3.2.2 Hardware Requirements 14
3.2.3 Dataset Requirements 15
3.3 System Design 16
3.3.1 Activity Diagram 16
3.3.2 Data flow Diagram 16
4 PROJECT DESCRIPTION 18
4.1 Methodology 18
4.2 Main Modules of the Project 19
4.2.1 Dual-Stream Transaction Ingestion 19
4.2.2 Privacy Preserving Transaction Processing 20
4.2.3 Duplicate Detection and Receipt Processing 20
4.2.4 Local-First Storage and Synchronization 20
4.2.5 Spending Analysis and Financial Guidance 21
4.3 Machine Learning Algorithm Comparison 22
4.3.1 Transaction Categorization 22
4.3.2 Spending Forecasting 22
4.3.3 Anomaly Detection and Segmentation 23
5 IMPLEMENTATION AND RESULTS 24
5.1 Proposed Implementation 24
5.1.1 Frontend Implementation 25

5.1.2 Backend and API Implementation 27

5.1.3 Data Preprocessing 28

5.2 Evaluation and Result Discussion 30

5.2.1 Dataset Analysis 30

5.2.2 Visual Analytics 31

5.2.1 Results 32

6 CONCLUSION AND FUTURE WORK 35

6.1 CONCLUSION 35

6.2 FUTURE WORK 36

APPENDIX 1 38

REFERENCES 40

LIST OF TABLES

TABLE NO.	TABLE NAME	PAGE NO.
2.1	Systematic Architectural Comparison of PFM Paradigms	11
3.1	Software requirements of the proposed system	14
3.2	Hardware requirements of the proposed system	14
5.1	Ingestion & Vision Accuracy	31
5.2	Latency	31

LIST OF FIGURES

FIGURE NO.	FIGURE NAME	PAGE NO.
3.1	Architecture of the Fintor Application	13
3.2	Activity diagram for transaction intelligence workflow	16
3.3	Data flow diagram of the proposed system	17
4.1	Processing Flow	21
5.1	App's Home Page	24
5.2	Backend and API Architecture	25
5.3	Data Preprocessing	27
5.4	Dataset	28
5.5	Category-wise expense distribution chart	29
5.6	Weekly or monthly spending trend	30

LIST OF ABBREVIATIONS

ABBREVIATION	EXPANSION
API	Application Programming Interface
CSV	Comma-Separated Values
EMI	Equated Monthly Installment
LLM	Large Language Model
MAE	Mean Absolute Error
ML	Machine Learning
NLP	Natural Language Processing
RMSE	Root Mean Squared Error
REST	Representational State Transfer
R^2	Coefficient of Determination
SQL	Structured Query Language
UPI	Unified Payments Interface
UI	User Interface

CHAPTER 1

INTRODUCTION TO SMART PERSONAL FINANCE MANAGEMENT

1.1 GENERAL

Digital payments have become a regular part of everyday financial activity. People now make payments through UPI, debit and credit cards, mobile banking applications, online subscriptions, and electronic fund transfers. While these methods make payments faster and more convenient, they also produce a large amount of transaction data. For an individual, keeping track of this information manually can become difficult, particularly when many low-value transactions occur throughout the month. Individually, these expenses may appear insignificant, but together they can form noticeable spending patterns.

Most personal finance applications help users record transactions, view category-wise expenses, monitor monthly spending, and set budgets. These features are useful for understanding past financial activity, but they generally focus on reporting what has already happened. This project explores a more analytical approach in which transaction data is used to support day-to-day financial decisions. Instead of only displaying expense totals, the proposed system aims to interpret transaction descriptions, learn from corrections made by the user, identify recurring spending patterns, explain changes in expenditure, estimate future spending, and provide early indications of possible budget-related risks.

One of the main difficulties in automated expense analysis is that transaction descriptions are not always clear. This is particularly relevant to UPI transactions in India, where a payment record may contain a person's name, a UPI ID, a mobile number, or another identifier instead of a recognizable merchant name. The meaning of such a transaction also depends on the individual user. For example, a recurring payment to the same person could represent rent, a personal transfer, or a payment to a small local business. As a

result, relying only on fixed keywords or globally defined merchant categories may lead to incorrect classification. To address this problem, the proposed platform follows a hybrid transaction intelligence approach that combines predefined rules, merchant normalization, machine learning, confidence-based classification, optional AI assistance, and feedback from the user.

In this project, financial wellness is considered from the perspective of understanding spending behaviour and using available financial information to make better-informed decisions. The proposed platform therefore functions as a personal financial analytics and decision-support system rather than a financial service provider. It does not perform transactions, transfer funds, generate credit scores, or present its outputs as certified financial advice. Its purpose is to organize transaction information, identify useful patterns, and present insights that can help users understand their financial behaviour.

1.2 OBJECTIVES

The primary objective of Fintor is to develop and evaluate a mobile personal financial management platform that automates transaction tracking, reduces manual expense entry, and supports users in understanding their spending behaviour while keeping sensitive financial information protected. The specific objectives of the project are:

1. **Automated Transaction Ingestion:** To develop an automated transaction-capture mechanism that can collect financial information from banking SMS messages and payment application notifications without requiring users to manually enter every transaction.
2. **Privacy-Preserving Local Processing:** To process sensitive transaction information on the user's device and remove unnecessary authentication-related information before the data is stored or synchronized, thereby reducing the exposure of personal financial data.
3. **Intelligent Transaction Parsing and Classification:** To extract important transaction attributes such as amount, transaction type, merchant or recipient, and

date, and classify expenses using rules, merchant normalization, machine learning, and user feedback.

4. **Duplicate Detection and Receipt Processing:** To identify duplicate transactions received through multiple sources and provide an on-device OCR mechanism for converting information from physical receipts and cash expense records into structured digital transactions.
5. **Offline-First Data Management:** To maintain transaction records locally so that core expense-tracking functions remain available without continuous network connectivity, while supporting secure and reliable synchronization with remote storage when a connection is available.
6. **Predictive Spending Analysis and Financial Guidance:** To analyse historical spending patterns, estimate future category-wise expenditure, identify potential budget overruns in advance, and provide rule-based financial guidance based on spending behaviour and available financial reserves.

1.3 EXISTING SYSTEM

The functional baseline for this project is provided by conventional personal expense management applications. These systems generally support features such as transaction detection, extraction of basic transaction details, expense categorization, transaction-history storage, spending charts, budget monitoring, and manual correction of incorrectly classified expenses. In mobile-based applications, transaction information can be collected from banking SMS messages or payment notifications. A typical processing flow can be represented as:

Bank or Merchant → Transaction Notification/SMS → Transaction Detection → Data Parsing → Local/Cloud Storage → Expense Categorization → Spending Analytics

Such systems provide users with a convenient way to review their financial activity, but their effectiveness depends on the quality and consistency of the transaction information they receive. Changes in message formats, incomplete merchant descriptions, and ambiguous payment identifiers can make automated classification more difficult.

Applications that rely on centralized synchronization may also require network connectivity when users access their information across multiple devices.

Another limitation is that basic expense dashboards mainly describe past spending through totals, categories, and charts. While this information helps users understand where their money has been spent, it may provide limited support for identifying unusual spending patterns or anticipating future budget requirements. This creates an opportunity for a system that goes beyond transaction recording and incorporates user-specific learning, predictive spending analysis, and proactive financial insights.

1.4 PROPOSED SYSTEM

The proposed system, Fintor, is a mobile personal financial management platform designed to automate expense tracking and provide users with meaningful insights into their spending behaviour. The system collects transaction information from banking SMS messages, payment application notifications, and physical receipts, reducing the need for manual expense entry. The captured information is processed on the device to extract transaction details such as amount, transaction type, merchant or recipient, and date. A combination of transaction rules, merchant normalization, machine learning, and user feedback is used to improve expense categorization, particularly for ambiguous transaction descriptions.

Fintor follows an edge-first and offline-first architecture in which sensitive transaction processing and local storage are performed primarily on the user's device. Duplicate transactions from different sources are identified before storage, while receipt information is converted into structured expense records using on-device OCR. Local SQLite storage allows the application to remain functional without continuous network connectivity, with MongoDB synchronization used when a network connection is available. Beyond recording expenses, the system analyses spending patterns, estimates future expenditure, identifies potential budget overruns, and provides rule-based financial guidance. The overall aim is to move personal expense management from simple transaction recording toward automated, personalized, and privacy-aware financial decision support.

1.5 SCOPE OF THE PROJECT

The scope of Fintor covers the automated processing and analysis of personal financial transaction data collected from multiple sources. The system focuses on extracting transaction details, categorizing expenses, normalizing ambiguous merchant or UPI identifiers, and maintaining a structured transaction history. It also includes spending analytics, individual behaviour analysis, overspending detection, identification of unusual spending patterns, future expenditure forecasting, and estimation of safe-to-spend amounts. Additional functionality includes financial goal tracking, personalized insights and recommendations, and visual representations of spending patterns through charts and dashboards. The project also covers local transaction processing, receipt-based expense capture, duplicate transaction detection, offline data storage, and synchronization with remote storage. These components are intended to support automated and privacy-aware expense management rather than requiring continuous manual entry.

1.6 ORGANIZATION OF THE REPORT

Chapter 1 introduces the problem domain and presents the motivation, objectives, existing approach, proposed system, problem statement, and scope of the project. Chapter 2 reviews relevant literature related to personal financial management, automated transaction processing, expense categorization, receipt processing, spending analysis, and financial forecasting, followed by a discussion of the identified research gaps. Chapter 3 presents the overall system architecture, functional and non-functional requirements, major system components, and database design. Chapter 4 describes the methodology used in Fintor, including transaction ingestion, data preprocessing, transaction classification, merchant normalization, duplicate detection, receipt processing, spending analysis, forecasting, and financial guidance. Chapter 5 discusses the system implementation and experimental evaluation, including the datasets, evaluation metrics, performance analysis, and results obtained from the implemented modules. The final chapter summarizes the contributions of the project, discusses its limitations, and outlines possible directions for future development.

CHAPTER 2
LITERATURE SURVEY ON SMART FINANCE MANAGEMENT

2.1 REVIEWS ON FINANCE MANAGEMENT

Personal Financial Management (PFM) systems have developed from manual expense recording into applications that can automatically collect, categorize, and analyse transaction data. Recent research has explored mobile expense tracking, automated extraction from financial messages, semantic transaction classification, receipt OCR, and predictive spending analysis. These developments provide the foundation for Fintor, while also highlighting areas where transaction interpretation, privacy, and proactive analysis can be further combined.

2.1.1 Bank Credential Aggregation and Screen-Scraping Models

Existing PFM applications generally provide transaction tracking, expense categorization, budgeting, financial summaries, and visualization [2], [4]. Research on mobile PFM applications has shown that automated transaction collection can reduce the effort associated with maintaining financial records [2], [6]. Recent systems have also explored offline expense tracking and automated categorization to improve usability [3], [7]. However, the effectiveness of automated PFM systems depends strongly on the quality of the available transaction information. Inconsistent transaction descriptions, incomplete merchant information, and differences between financial institutions can reduce classification accuracy [5], [7]. Account aggregation through bank APIs or external platforms can reduce manual transaction entry and provide information from multiple financial institutions through a common interface. However, this approach depends on external services, network connectivity, and the authentication mechanisms supported by individual financial institutions.

Therefore, the proposed system aims to obtain useful transaction information directly from the user's device, reducing dependence on external account aggregation services and avoiding the need to process the user's complete financial history remotely.

2.1.2 SMS-Based Transaction Extraction and Mobile Notifications

Financial SMS messages have been widely investigated as a source of structured transaction information [5], [6]. Recent research has applied natural language processing and named-entity recognition techniques to extract transaction amounts, account information, transaction types, and other attributes from financial messages [5]. These approaches demonstrate that semi-structured banking messages can be converted into useful transaction records without requiring manual entry [5], [6]. However, relying only on SMS messages may result in incomplete transaction records because Android payment applications can also generate notifications containing important transaction details.

Therefore, Fintor adopts a multi-source approach by combining financial SMS messages with application notifications accessed through the Android NotificationListenerService, with the appropriate user permission [8]. The captured information is processed locally and converted into a normalized transaction structure.

2.1.3 Transaction Classification and Merchant Normalization

Automatic expense categorization remains an important challenge in PFM systems because transaction descriptions do not always clearly identify the nature of a purchase. Recent research has investigated semantic classification and machine-learning methods for identifying expense categories beyond simple keyword matching, particularly when transaction descriptions contain varied or ambiguous language [5], [7]. This issue is especially relevant to UPI transactions, where a transaction description may contain a person's name, UPI identifier, or other payment information rather than a conventional merchant name [9]. Furthermore, the same identifier may represent different types of transactions depending on the individual user and the transaction context.

Therefore, Fintor treats transaction classification as a hybrid problem by combining predefined rules, merchant normalization, machine-learning classification, confidence assessment, and user corrections. This approach allows the system to adapt its

interpretation based on the user's transaction history instead of relying exclusively on a fixed global category mapping [4], [7].

2.1.4 Receipt OCR and On-Device Processing

Physical receipts remain an important source of expense information when transactions are made using cash or when digital transaction records do not contain sufficient descriptive details. However, manually entering information from paper receipts can be time-consuming and may lead to incomplete financial records. Research on mobile receipt processing has explored image detection, text extraction, and structured information recovery, demonstrating the potential of computer vision and optical character recognition to convert unstructured receipts into machine-readable financial records [10].

Therefore, Fintor incorporates on-device OCR to extract relevant information from physical receipts and integrate it with digitally captured transactions. Google ML Kit provides text-recognition capabilities that can operate directly on mobile devices [11]. This local processing approach reduces the need to upload receipt images to remote services, which is particularly important because receipts may contain sensitive information such as merchant details, addresses, and transaction amounts.

2.1.5 Spending Analysis and Financial Forecasting

Traditional PFM applications mainly focus on recording past transactions and presenting historical spending summaries. Although these features help users understand previous financial activity, they may not provide early indications of future expenditure or potential budget risks. Recent research has therefore explored forecasting and predictive-analysis techniques that use historical category-level spending to identify trends and estimate expenditure for subsequent periods [13], [14]. Such methods can transform a PFM application from a retrospective reporting tool into a system that supports more informed financial decisions.

Therefore, Fintor uses category-level spending history to estimate spending velocity and projected expenditure for the current period. This information helps identify categories that are approaching their assigned budgets and enables the system to provide early notifications. The forecasts are presented as decision-support information rather than guaranteed predictions because actual expenditure may vary according to user behaviour and unexpected financial events [13].

2.1.5 Budgeting and Financial Decision Support

Budgeting frameworks such as the 50/30/20 rule provide general guidelines for dividing income among needs, wants, and savings. However, individual financial circumstances and spending patterns can differ considerably. Consequently, a fixed budgeting rule alone may not adequately represent a user's actual spending behaviour, recurring obligations, financial goals, or available reserves. Research on personalized financial insights and behaviour-based expense analysis supports the need for budgeting approaches that consider individual spending patterns [4], [13], [14].

Therefore, Fintor treats budgeting as a combination of historical spending analysis, category limits, financial goals, and available reserves rather than relying on a single allocation rule. Financial guidance is presented as rule-based decision support to help users review their spending and savings behaviour. Investment execution and certified financial advice remain outside the scope of the system.

2.1.6 Research Gap

The reviewed literature demonstrates progress in individual areas such as automated personal financial management [2], [4], [6], financial SMS extraction [5], notification-based transaction capture [8], semantic expense classification [7], receipt OCR [10], [11], offline expense tracking [3], and spending forecasting [13], [14]. However, these capabilities are often investigated as separate components. A practical PFM platform must address how multiple transaction sources interact while also

managing ambiguous merchant information, duplicate records, privacy-sensitive processing, offline availability, synchronization, and predictive spending analysis.

Therefore, Fintor addresses this integration gap by combining multi-source transaction ingestion, local transaction processing, hybrid expense classification, merchant normalization, duplicate detection, on-device receipt OCR, offline-first storage, synchronization, and predictive spending analysis within a single personal financial management workflow [3], [5], [7], [11]–[14]. The contribution of the proposed system is centred on integrating and implementing these capabilities rather than claiming that any individual technique is novel in isolation.

2.2 COMPARISON AND DISCUSSION

The reviewed approaches reveal several practical challenges that remain relevant when designing an automated personal financial management system. First, transaction information can originate from different channels, including banking SMS messages and payment-application notifications. Relying on a single source can limit the information available to the application when transaction details are distributed across different notification mechanisms. Fintor therefore combines SMS-based and application-notification inputs so that transaction information from multiple sources can be processed within a common pipeline.

Second, receiving the same transaction through multiple sources can result in duplicate records. For example, a transaction may generate both a banking notification and a payment-application notification. Fintor addresses this problem through deterministic transaction fingerprinting that combines relevant transaction attributes with temporal information.

Third, existing PFM approaches commonly focus on recording and summarizing historical transactions, while predictive spending analysis can provide additional information about possible future budget requirements. Fintor calculates category-level spending velocity and uses the observed spending rate to estimate potential period-end expenditure. The resulting estimates are used to generate early budget warnings rather

than simply reporting an exceeded budget after the event. Financial guidance is similarly implemented as a rule-based decision-support component that considers spending behaviour and available reserves rather than automatically executing financial transactions.

Overall, the comparison indicates that automated transaction capture, expense classification, receipt processing, duplicate resolution, and spending prediction are individually established areas of research and application development. The research gap addressed by Fintor lies in bringing these capabilities together within a local-first PFM architecture. To highlight the architectural innovations of Fintor, Table 2.1 provides a detailed comparative matrix against traditional commercial and academic PFM paradigms.

Architectural Comparison

Table 2.1: Systematic Architectural Comparison of PFM Paradigms

Architectural Feature	Credential Aggregators (e.g., Plaid, Mint)	Commercial SMS Miners (e.g., Axio / Walnut)	Proposed Framework: Fintor
Ingestion Pipeline	Server-side web scraping & Open Banking APIs.	Single-channel cellular SMS BroadcastReceiver.	Dual-stream edge listener: SMS BroadcastReceiver + UPI NotificationListenerService.
Data Custody & Privacy	Bank credentials and raw histories stored on third-party cloud servers.	SMS telemetry uploaded to cloud for server-side credit scoring.	Local-first SQLite storage; volatile RAM scrubbing of OTPs/PII; zero cloud telemetry exfiltration.
Micropayment Capture (<₹500)	High coverage (dependent on bank sync delays).	Blind to micro-debits suppressed by cellular bank SMS networks.	Complete coverage: Intercepts real-time UPI app push broadcasts (GPay, PhonePe, Paytm).
Deduplication Mechanics	Server-side timestamp heuristics; prone to multi-sync duplicates.	Basic string matching; vulnerable to dual SMS alerts.	Deterministic Compound Hashing: Edge SHA-256 over 180s bucketing windows.

Non-Digital (Cash) Spends	None (requires manual entry).	None (requires manual keyboard entry).	On-Device Vision: Embedded MobileNet OCR parses paper receipts on CPU in $\sim 312ms$.
Grammar Adaptability	Dependent on aggregator API fixes.	Hardcoded app releases; breaks when bank SMS formats update.	Decoupled Dynamic Schema: In-memory regex compilation via dynamic JSON template fetching.
System Reactivity	High latency ($2 -- 10s$ cloud sync roundtrips).	Moderate latency (cloud-dependent category views).	Sub-15 ms UI latency via local embedded SQLite engine (sqflite).

2.3 CONCLUSION

The literature review shows that personal financial management has progressed from manual expense recording toward automated transaction capture, classification, document processing, and predictive analysis. However, these capabilities are often implemented as separate functions, leaving practical challenges related to ambiguous transaction descriptions, multiple transaction sources, duplicate records, privacy-sensitive processing, and proactive financial analysis.

Fintor addresses these challenges through an integrated edge-first architecture. The system combines multi-source transaction ingestion, local preprocessing, hybrid classification, deterministic duplicate detection, on-device receipt OCR, offline-first storage, synchronization, and spending analysis within a single workflow. Rather than replacing existing financial services or executing financial transactions, Fintor is designed as a personal decision-support platform that helps users organize transaction data, understand their spending behaviour, and identify potential budget risks earlier. This architecture provides the basis for the system design, implementation, and evaluation presented in the following chapters.

CHAPTER 3
PROPOSED SYSTEM - FINTOR

3.1 SYSTEM ARCHITECTURE OF FINTOR

The proposed architecture separates user interaction, API and authentication, application processing, transaction intelligence, analytics and prediction, and cloud persistence. This separation allows transaction ingestion and machine learning services to evolve independently while maintaining a consistent user-facing platform

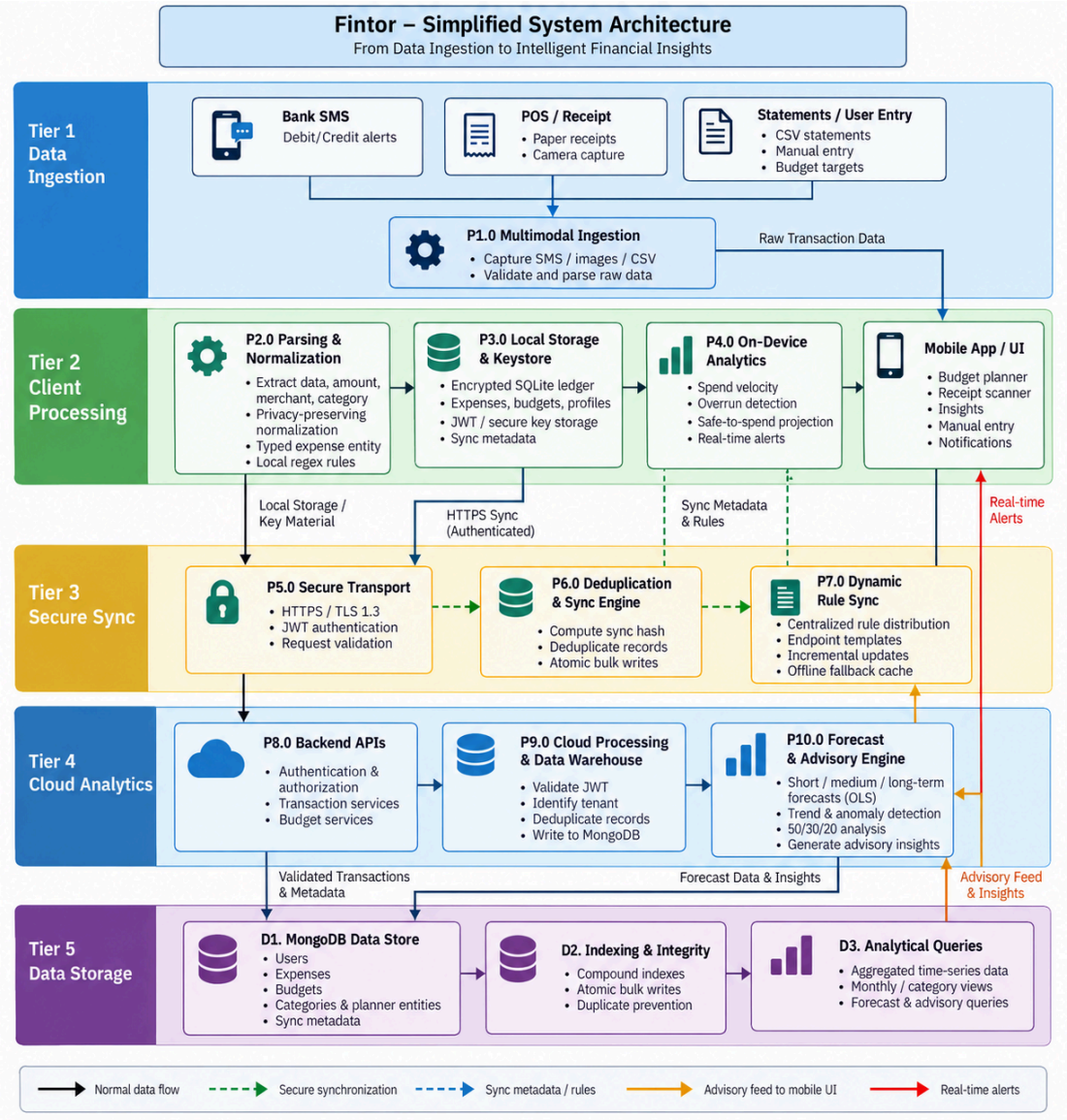


Figure 3.1. Architecture of the Fintor Application

3.2 SYSTEM REQUIREMENTS

3.2.1 Software Requirements

Table 3.1. Software requirements of the proposed system

Software Component	Suggested Technology	Purpose
Mobile Frontend	Flutter SDK with Dart VM	Cross-platform Android and iOS user interface
Client Local Database	SQLite (`sqflite` plugin)	Offline-first local persistence layer enabling instant sub-15 ms UI reactivity and offline functionality
Native Ingestion Service	Kotlin / Native Android	SMS
On-Device Vision Pipeline	Google ML Kit (MobileNet OCR)	On-device CPU-based OCR for offline physical receipt and cash memo digitization in ~312 ms
Backend Framework	Node.js with Express.js	Backend Server
Cloud Database	MongoDB Atlas with Mongoose	Distributed NoSQL document store
Authentication & Security	JSON Web Token (JWT) & bcrypt	Stateless bearer token authentication signed with HMAC-SHA256 and salted password hashing

3.2.2 Hardware Requirements

Table 3.2. Hardware requirements of the proposed system

Component	Suggested Requirement	Purpose
CPU	Modern multi-core processor	Application and data processing
RAM	8 GB minimum; higher capacity preferred for model experimentation	Data analysis and concurrent development
Storage	Sufficient SSD storage based on actual dataset size	Source code, datasets, models, and artifacts

3.2.3 Dataset used for Category Distribution

The proposed system requires transaction data that can support automated ingestion, expense classification, feature engineering, spending analysis, visualization, and predictive modelling. The core dataset may contain fields such as transaction ID, user ID, date, time, amount, debit or credit type, transaction type, transaction description, merchant name or identifier, UPI handle where available, payment method, category, and transaction status.

Additional behavioural features can be derived from these fields to support analysis and prediction. These may include transaction frequency, average transaction value, category-wise spending frequency, daily and monthly expenditure, rolling spending averages, recurring transaction patterns, budget utilization, and deviations from the user's usual spending behaviour.

The final dataset characteristics should be reported only after the dataset has been selected and prepared. These characteristics include the data source, number of records, number of users, category distribution, missing-value statistics, class balance, preprocessing steps, and training and testing strategy. If synthetic data are used for development or demonstration, the report should clearly identify them as synthetic and describe the method used to generate the records. This distinction is important when interpreting the performance of the machine-learning and forecasting components.

3.3 SYSTEM DESIGN

3.3.1 Activity Diagram for transaction intelligence and financial analytics

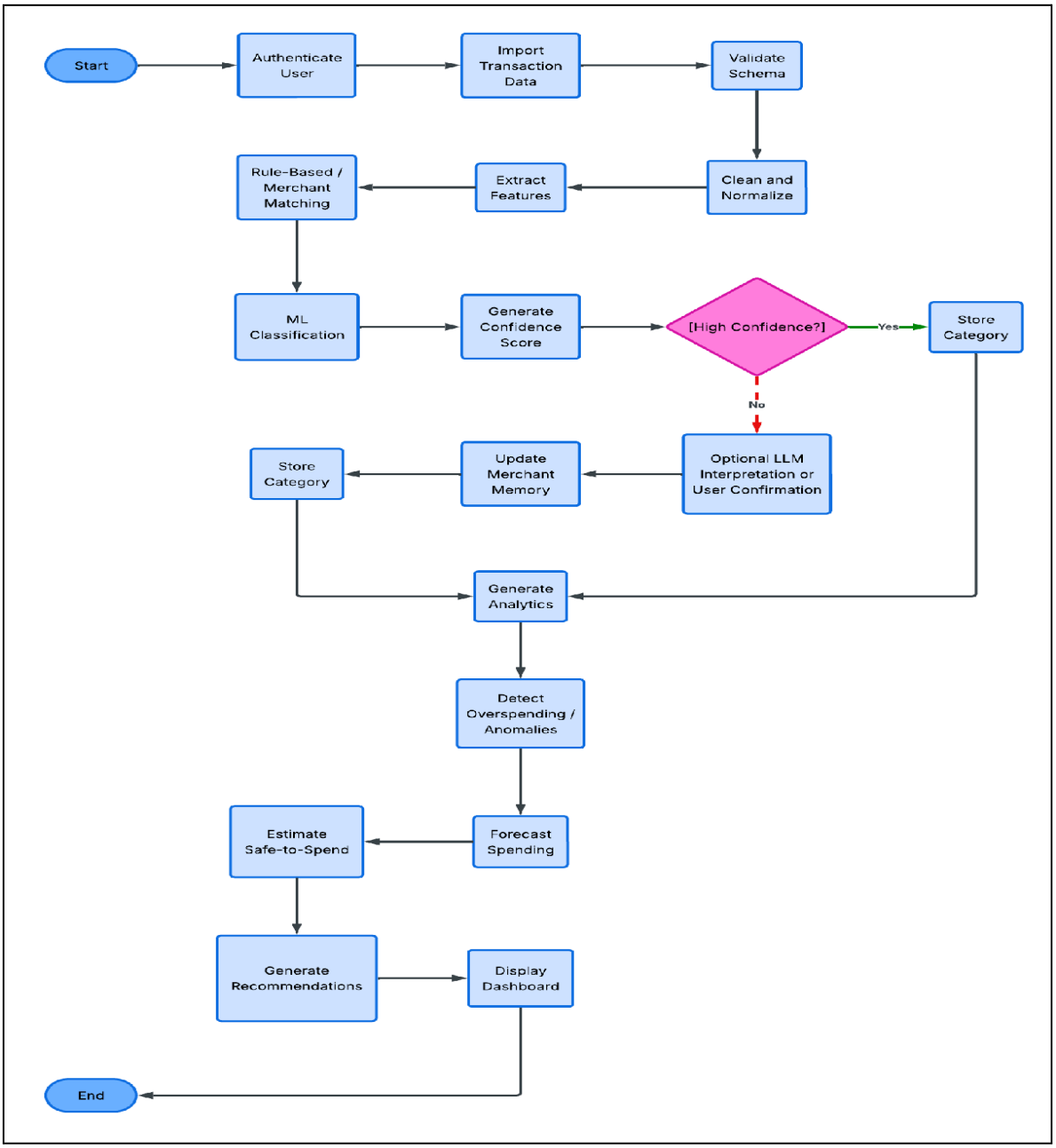


Figure 3.2. Activity flow for transaction intelligence and financial analytics

3.3.2 Data Flow Diagram

At the context level, the user provides transaction data and financial settings to the platform and receives categorized transactions, analytics, alerts, forecasts, safe-to-spend estimates, goal progress, and recommendations. At the first decomposition level, the major processes are transaction ingestion, preprocessing, categorization, feedback learning, analytics, prediction, and recommendation generation. Data stores include users, transactions, categories, budgets, goals, merchant memory, derived features, and prediction outputs.

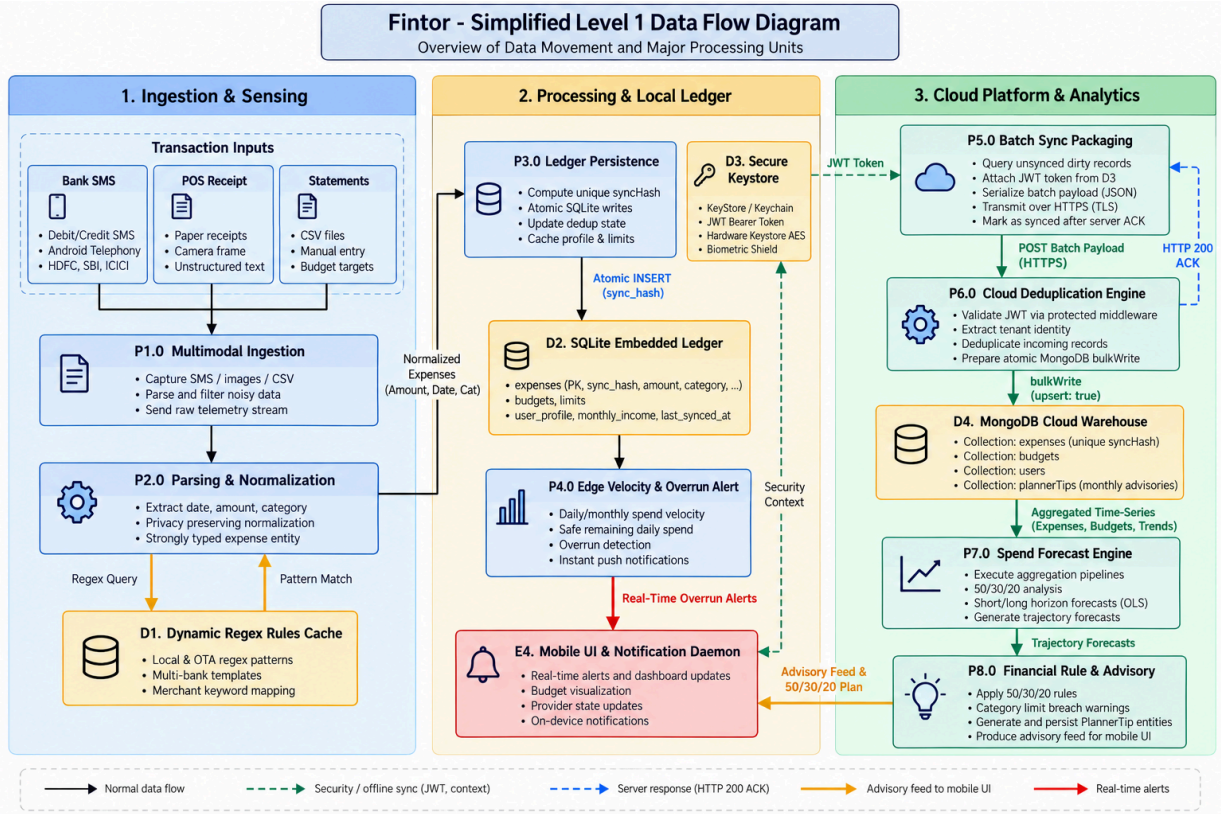


Figure 3.3. Data Flow Diagram

CHAPTER 4
PROJECT DESCRIPTION

4.1 METHODOLOGY

Fintor follows an edge-first and local-first methodology in which the major stages of financial data processing are performed on the user's mobile device before synchronization with remote storage. The methodology begins with automated transaction ingestion through two Android-based channels: a native BroadcastReceiver captures relevant banking SMS messages, while NotificationListenerService collects transaction notifications generated by payment applications such as Google Pay, PhonePe, and Paytm. The captured communication is first subjected to local privacy and noise filtering, where irrelevant messages and authentication-related content such as OTPs are removed before financial information is extracted. The transaction parsing engine then identifies key attributes including the monetary amount, debit or credit type, account identifier, merchant or recipient information, and timestamp. Configurable parsing templates allow the extraction rules to accommodate variations in message formats without requiring changes to the core application.

After extraction, transaction information is normalized to create a consistent internal representation. Merchant names and payment identifiers are processed to improve categorization, particularly for transactions where the original description may contain a personal name, UPI identifier, or other ambiguous information. A hybrid classification mechanism combines predefined rules, merchant normalization, machine-learning-based classification, confidence assessment, and corrections provided by the user. When the same financial event is received through multiple channels, Fintor applies deterministic transaction fingerprinting using relevant transaction attributes and temporal information to identify probable duplicates before persistence. Physical cash expenses and paper receipts are handled through an on-device OCR pipeline using Google ML Kit, which extracts relevant information such as merchant names, dates, and transaction amounts without requiring receipt images to be processed by a remote OCR service.

The processed transactions are stored locally in SQLite, enabling the application to access financial records without continuous network connectivity. Local storage also supports the separation of sensitive processing from the synchronization layer. When network connectivity is available, structured transaction records are synchronized asynchronously with MongoDB Atlas using idempotent database operations, allowing the remote component to function primarily as a persistence and synchronization layer. The methodology therefore maintains a local copy of the user's transaction history while supporting access to synchronized data across the application's supported environments.

Once transaction records have been structured and stored, Fintor performs spending analysis using historical category-level expenditure. Category spending velocity is calculated as vc=Sc/tv_c=S_c/t, where ScS_c represents the observed spending for category cc and tt represents the elapsed observation period. After sufficient transaction history has been accumulated, the resulting spending rate is used to estimate potential period-end expenditure and identify categories that may approach or exceed their assigned budgets. The system generates proactive warnings based on these estimates rather than waiting until a budget has already been exceeded. A rule-based financial guidance component can additionally consider spending behaviour, available reserves, financial goals, and predefined budgeting conditions when generating recommendations. These recommendations are presented as decision-support information and do not execute financial transactions or represent certified financial advice. Overall, the methodology integrates automated multi-source transaction capture, local preprocessing, transaction classification, duplicate detection, receipt digitization, offline-first storage, synchronization, and predictive spending analysis into a single financial management workflow.

4.2 MAIN MODULES OF THE PROJECT

4.2.1 Dual-Stream Transaction Ingestion

Fintor collects transaction information through two complementary sources: banking SMS messages captured using Android's BroadcastReceiver and payment-application notifications captured using NotificationListenerService. This approach allows the system to process both conventional banking alerts and notifications generated by payment

applications such as Google Pay, PhonePe, and Paytm. The captured information is passed to the local processing pipeline rather than requiring users to manually enter individual transactions.

4.2.2 Privacy-Preserving Transaction Processing

Before transaction information is stored, the system performs local preprocessing to remove irrelevant or sensitive information such as authentication-related messages and OTP content. A configurable parsing engine then identifies important transaction fields including amount, debit or credit status, merchant or recipient information, account identifiers, and timestamps. Using configurable patterns allows the parser to accommodate differences in transaction formats without requiring major changes to the application.

4.2.3 Duplicate Detection and Receipt Processing

Transactions received from multiple sources may represent the same financial event. Fintor therefore generates deterministic transaction fingerprints using relevant transaction attributes and combines them with temporal information to identify likely duplicates before persistence. The system also supports physical expense records through on-device receipt processing using Google ML Kit. Relevant text such as merchant names, dates, and amounts is extracted from receipts and converted into structured transaction data.

4.2.4 Local-First Storage and Synchronization

Processed transactions are stored locally using SQLite, allowing the core application to operate without continuous network connectivity. Local storage also provides faster access to recent transaction information and supports offline expense tracking. When network connectivity is available, structured records are synchronized with MongoDB Atlas through asynchronous and idempotent database operations. The cloud component therefore acts primarily as a synchronization and persistence layer rather than the main processing environment.

4.2.5 Spending Analysis and Financial Guidance

Fintor analyses accumulated transaction data to identify spending patterns and estimate future expenditure. Category spending velocity is represented as vc=Sc/t where Sc represents spending in category cc during the observed period and tt represents the elapsed time. This measure is used to estimate potential month-end spending and identify categories that may exceed their assigned budgets. The system can then generate early warnings and provide rule-based financial guidance based on spending behaviour and available financial reserves. The overall processing flow can be summarized as shown in the figure 4.1.



Figure 4.1 Processing Flow

4.3 MACHINE LEARNING AND ALGORITHM COMPARISON

4.4.1 Transaction Categorization

Candidate models include Logistic Regression, Random Forest Classifier, and XGBoost Classifier. Additional models such as Naive Bayes, Support Vector Machine, or neural networks may be considered only if justified by the dataset and implementation. The final model should be selected using actual experimental results. Accuracy, precision, recall, and F1 score are proposed evaluation metrics.

Table 4.3. Transaction categorization algorithm comparison

Algorithm	Accuracy	Precision	Recall	F1 Score
Logistic Regression	0.9217	0.9299	0.9217	0.9223
Random Forest Classifier	0.8172	0.8429	0.8172	0.8107
XGBoost Classifier	0.9389	0.9401	0.9389	0.9391

4.4.2 Spending Forecasting

Candidate forecasting models include Linear Regression, Random Forest Regressor, and XGBoost Regressor. If the dataset contains sufficient time-series structure, ARIMA or another suitable time-series approach may also be evaluated. Model comparison should use actual MAE, RMSE, and R^2 values.

Table 4.4. Spending forecasting algorithm comparison

Algorithm	MAE	RMSE	R^2
Linear Regression	1070.61	1647.27	0.0328
Random Forest Regressor	905.49	1338.81	0.3512
XGBoost Regressor	875.65	1277.40	0.3926

4.4.3 Anomaly Detection & Behavioural Segmentation

In future versions, we plan to add **anomaly detection** using techniques such as Isolation Forest, Local Outlier Factor, or One-Class SVM to identify unusual spending patterns. This will focus on detecting deviations from normal behaviour rather than claiming complete fraud detection. We also plan to introduce **behavioural segmentation** using K-Means clustering to identify spending groups based on actual user data and patterns. These features will be evaluated using suitable datasets and metrics such as precision, recall, F1 score, and false positive rate before being integrated into the system.

CHAPTER 5
IMPLEMENTATION AND RESULTS

5.1 PROPOSED IMPLEMENTATION

Fintor is implemented as a cross-platform, multi-tier system consisting of a Flutter-based mobile application with native Android components and a Node.js/MongoDB backend responsible primarily for synchronization and persistent storage. On the mobile side, transaction information is collected through two Android mechanisms. A native Kotlin BroadcastReceiver captures relevant banking SMS messages, while NotificationListenerService receives notifications generated by supported payment applications such as Google Pay, PhonePe, and Paytm. The captured messages are passed through local preprocessing to remove irrelevant and authentication-related content before transaction fields are extracted using configurable regular-expression templates. These templates are maintained separately from the application logic, allowing parsing rules to be updated without modifying the core transaction-processing implementation.

For physical expenses, Fintor provides receipt-based transaction capture using Google ML Kit text recognition. The OCR pipeline processes receipt images on the device and extracts relevant information such as merchant details, dates, and transaction amounts, allowing cash expenses and other physical records to be incorporated into the same transaction workflow. Transactions obtained from different sources are then normalized and subjected to deterministic duplicate detection using a SHA-256-based fingerprint that combines relevant transaction attributes with temporal information. This prevents repeated notifications representing the same financial event from being stored as separate records.

After preprocessing and validation, transaction data is persisted locally using SQLite through the sqflite package. This local-first design allows the application to access previously processed financial records without requiring continuous network connectivity. When connectivity is available, pending records are synchronized asynchronously with MongoDB Atlas through the Node.js REST backend using idempotent database operations. The synchronization layer therefore maintains a remote copy of the structured

transaction data without making the cloud backend responsible for the primary transaction-processing workflow.

The final implementation layer performs spending analysis directly from the structured transaction history. A Dart-based rule engine evaluates category-level spending rates, budget utilization, and available financial reserves to identify potential overspending conditions and generate user-facing alerts. The system can also apply predefined financial-planning rules when evaluating surplus funds and emergency reserves. These outputs are presented as personalized decision-support information rather than automated financial transactions or guaranteed investment recommendations. Overall, the implementation connects native Android data-capture services, local transaction processing, SQLite storage, receipt OCR, predictive spending analysis, and asynchronous cloud synchronization into a unified personal financial management application.

5.1.1 FRONTEND IMPLEMENTATION

The Fintor frontend is implemented using Flutter to provide a cross-platform mobile interface with a consistent user experience. The application is organized into reusable screens and components for transaction tracking, expense categorization, budgeting, financial goals, spending analytics, and personalized insights. Dashboard views present key financial information through summary cards, charts, category-wise breakdowns, and budget indicators. User interactions such as transaction review, category correction, goal updates, and budget configuration are handled through responsive form components. Local SQLite data is accessed through the application layer, allowing transaction records and analytics to remain available during offline operation. Native Android services are integrated with Flutter where platform-specific functionality is required.

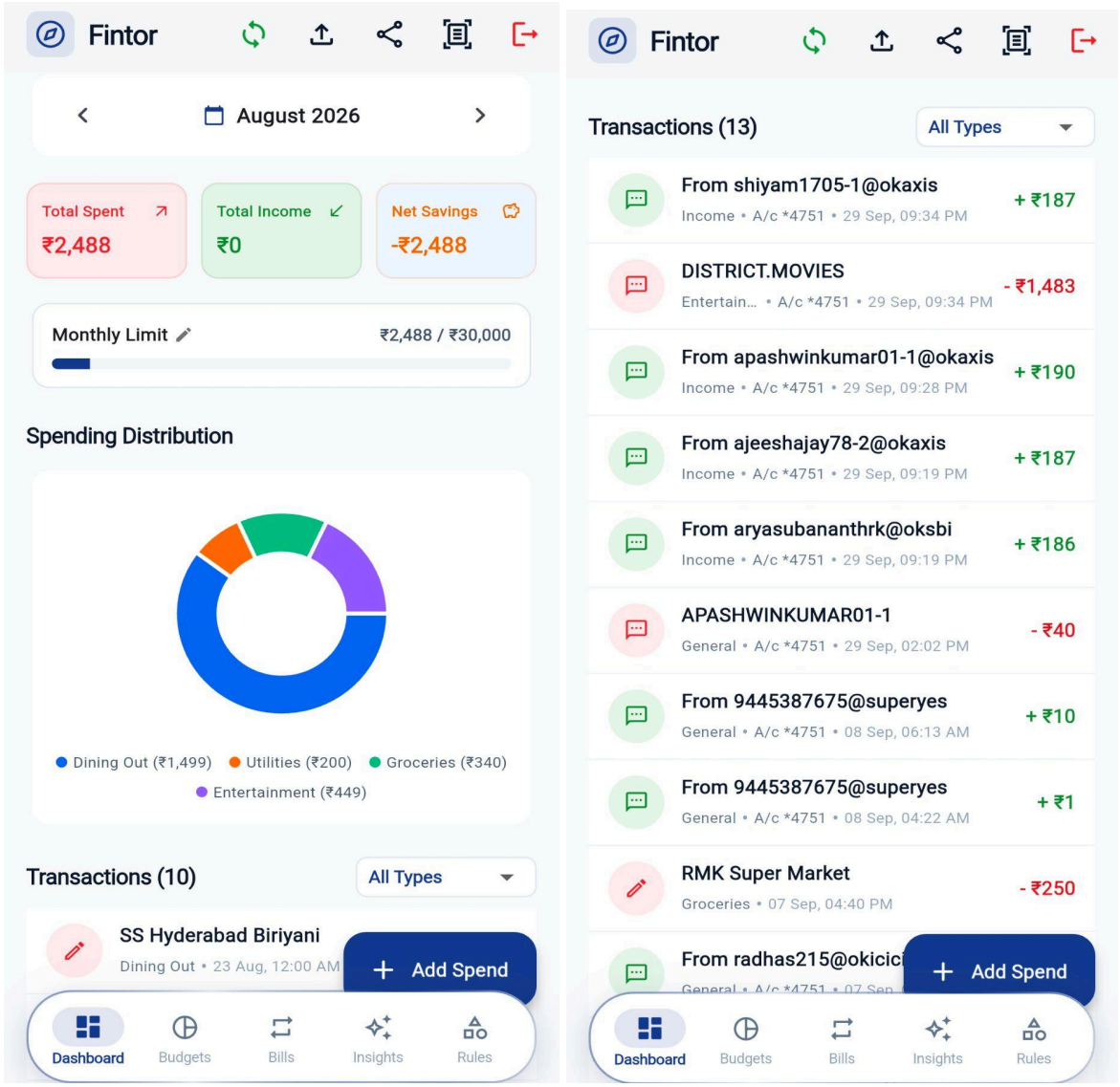


Figure 5.1 Fintor Application Home Dashboard

5.1.2 BACKEND AND API IMPLEMENTATION

The Fintor backend is developed using Node.js and Express.js, with MongoDB Atlas providing cloud-based persistent storage. It exposes RESTful APIs for authentication, configuration management, and synchronization of processed transaction data. Authentication uses JSON Web Tokens (JWT), while passwords are protected using bcrypt hashing. The backend provides a template-distribution endpoint that delivers configurable SMS parsing patterns to mobile clients, allowing transaction formats to be updated independently of the application release. A batch synchronization endpoint receives sanitized transaction records from the mobile application and performs idempotent MongoDB bulk operations using transaction fingerprints and unique indexing. This approach supports reliable synchronization, prevents duplicate records during retries, and maintains consistency across supported devices.

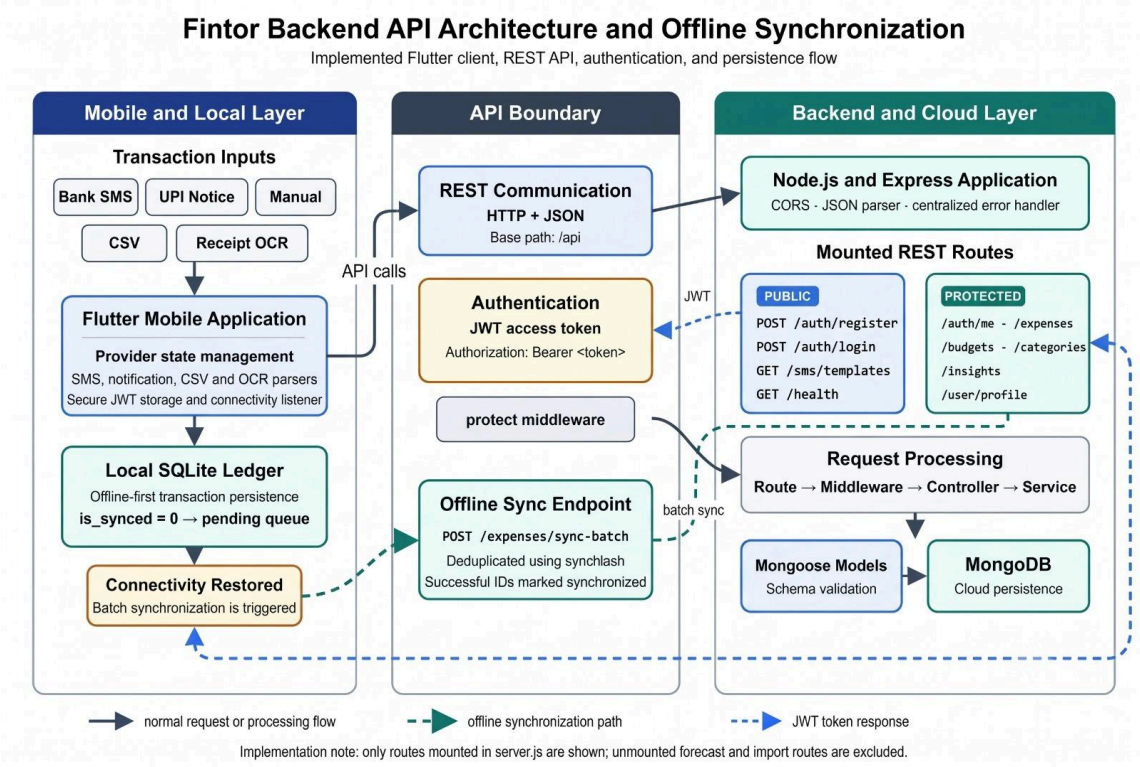


Figure 5.2 Backend and API Architecture

5.1.3 DATA PREPROCESSING IMPLEMENTATION

The data preprocessing framework in Fintor converts raw financial communication and receipt data into standardized transaction records through a predominantly on-device processing pipeline. Banking SMS messages are captured using an Android BroadcastReceiver, while payment notifications from supported applications such as Google Pay, PhonePe, and Paytm are collected through NotificationListenerService. The captured content is locally filtered to remove irrelevant and authentication-related information before transaction attributes are extracted. A configurable regular-expression engine then identifies key fields including transaction amount, debit or credit type, account suffix, merchant or recipient identifier, and timestamp. The parsing patterns are maintained through configurable JSON templates, allowing message-format changes to be handled independently of the application logic.

Physical receipts are processed through Google ML Kit text recognition on the device, enabling information such as merchant names, dates, and amounts to be extracted without requiring receipt images to be processed remotely. Records obtained from digital and physical sources are normalized into a common transaction structure before duplicate detection is performed. Fintor generates a deterministic SHA-256 transaction fingerprint using selected transaction attributes and temporal information to identify records that are likely to represent the same financial event. Validated records are then stored in the local SQLite database, allowing offline access to transaction information. Pending records are subsequently synchronized with MongoDB Atlas through asynchronous and idempotent batch operations when network connectivity is available.

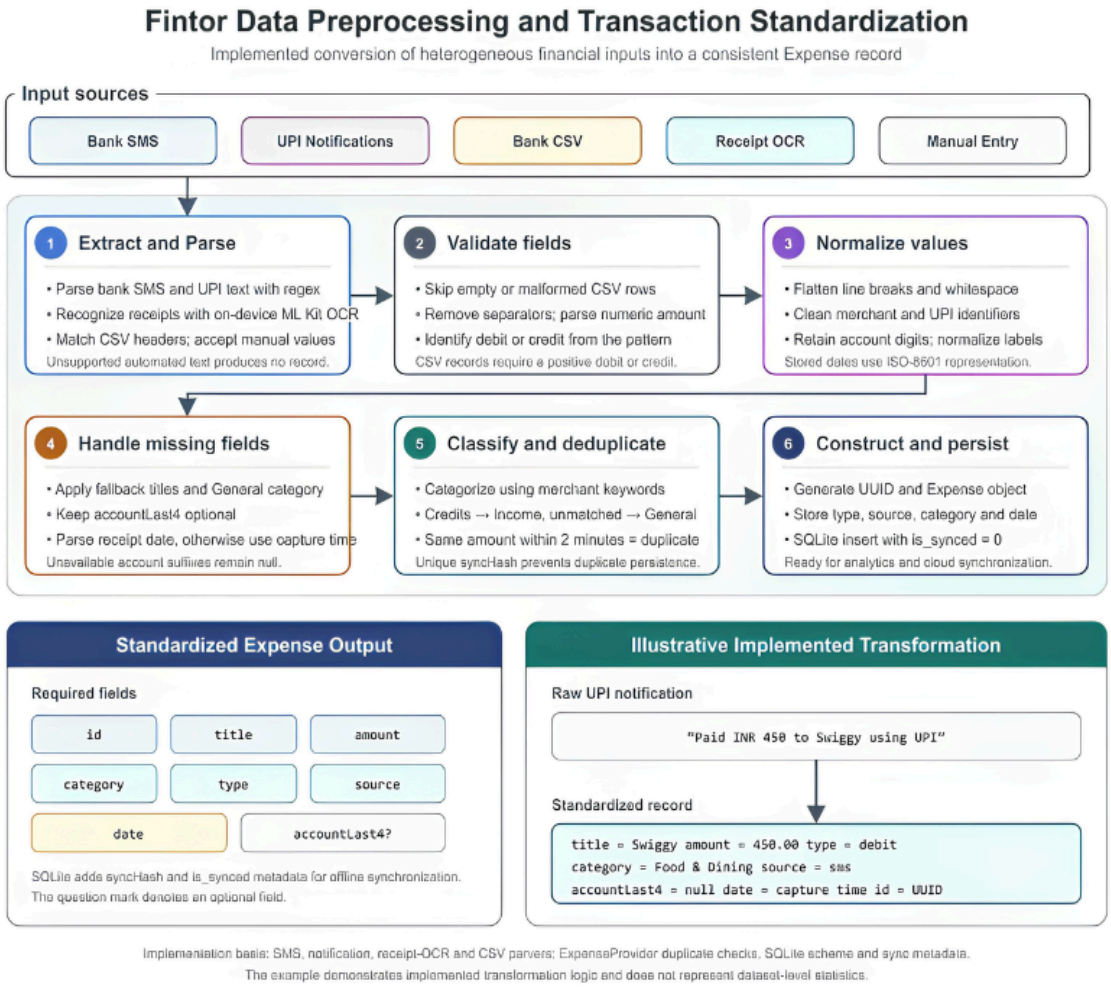


Figure 5.3 Data Preprocessing

Fintor uses a four-tier local categorization pipeline to assign standardized transactions to spending categories while keeping the primary classification process on the user's device. The process begins with entity and VPA normalization, where transaction prefixes, unnecessary characters, and banking-specific formatting are removed to isolate useful merchant or recipient identifiers. For example, a descriptor such as `UPI-SWIGGY-1293@icici` can be normalized to a representative merchant identifier such as `SWIGGY`. The normalized identifier is then evaluated through four classification stages. The first stage checks locally stored user overrides, allowing previous manual corrections to take priority over general classification rules. If no user-specific rule is available, the second stage performs exact and regular-expression matching against a configurable merchant dictionary. The third stage applies keyword-based heuristics to

identify sector-related terms within transaction descriptions and VPAs. The final stage uses a lightweight local classification mechanism based on techniques such as term-frequency analysis, Naive Bayes, or similarity matching to handle less structured merchant descriptions, local businesses, and OCR-derived transaction text. Each classification result is associated with a confidence level. Transactions that cannot be classified reliably are assigned to a general category and presented to the user for confirmation. When the user corrects a category, the correction can be stored as a local override rule and subsequently used when similar transactions are encountered. This feedback mechanism allows the categorization process to adapt to user-specific spending patterns without requiring transaction data to be sent to an external classification service.

5.2 EVALUATION AND RESULT DISCUSSION

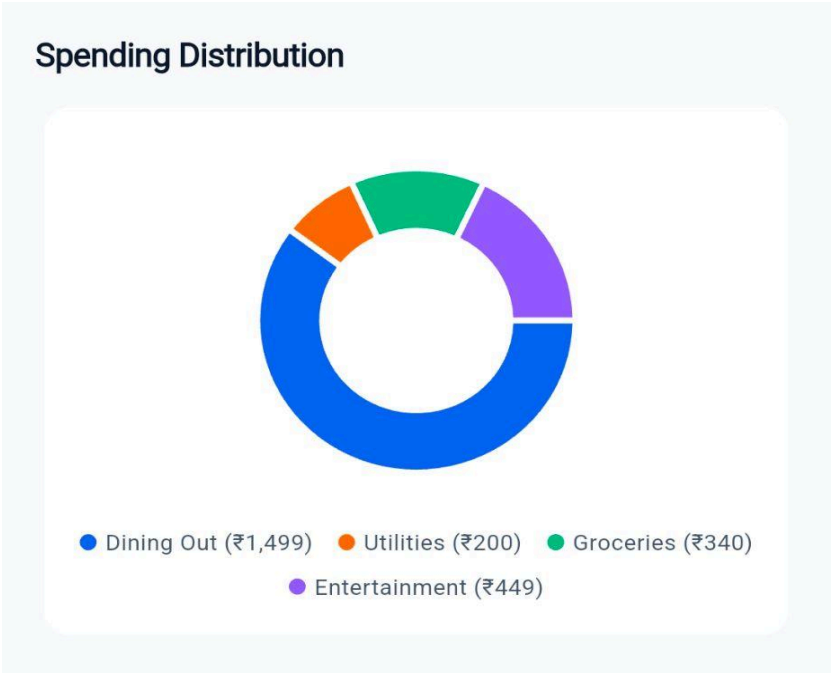
This section is intentionally structured for completion after implementation. Dataset statistics, visualizations, model performance values, confusion matrices, error measures, anomaly validation, and screenshots must be inserted from actual experiments. The report should not replace missing experiments with assumed values.

5.2.1 DATASET ANALYSIS

	A	B	C	D	E	F	G	H	I
1	transactio	transactio	descriptior	amount_ir	currency	matched_l	category	label_sour	is_synthet
2	TXN00362	########	UPI/steam	1160.8	INR	steam	Entertainn	keyword_r	TRUE
3	TXN00074	########	CARD/burg	1775.72	INR	burger	Food & Dir	keyword_r	TRUE
4	TXN00375	########	NETBANKI	2349.33	INR	inox	Entertainn	keyword_r	TRUE
5	TXN00156	########	Rapido pay	4330.77	INR	rapido	Travel & Fi	keyword_r	TRUE
6	TXN00105	########	DEBIT CAR	1964.06	INR	shell	Travel & Fi	keyword_r	TRUE
7	TXN00395	########	PLAYSTATI	1119.45	INR	playstatior	Entertainn	keyword_r	TRUE
8	TXN00378	########	STEAM ON	2464.02	INR	steam	Entertainn	keyword_r	TRUE
9	TXN00125	########	NETBANKI	2551.01	INR	metro	Travel & Fi	keyword_r	TRUE
10	TXN00069	########	STARBUCK	1107.35	INR	starbucks	Food & Dir	keyword_r	TRUE
11	TXN00451	########	VI ONLINE	2370.56	INR	vi	Utilities &	keyword_r	TRUE
12	TXN00010	########	Starbucks	2065.71	INR	starbucks	Food & Dir	keyword_r	TRUE
13	TXN00195	########	Makemytr	2210.02	INR	makemytri	Travel & Fi	keyword_r	TRUE
14	TXN00407	########	UPI-tneb 4	6676.54	INR	tneb	Utilities &	keyword_r	TRUE
15	TXN00085	########	IMPS-cafe	1152.64	INR	cafe	Food & Dir	keyword_r	TRUE
16	TXN00372	########	PLAYSTATI	1694.21	INR	playstatior	Entertainn	keyword_r	TRUE
17	TXN00389	########	CARD/netf	1482.74	INR	netflix	Entertainn	keyword_r	TRUE
18	TXN00496	########	POS VI TXN	2911.7	INR	vi	Utilities &	keyword_r	TRUE
19	TXN00031	########	UPI/KFC H	1933.91	INR	kfc	Food & Dir	keyword_r	TRUE
20	TXN00317	########	POS inox 8	587.88	INR	inox	Entertainn	keyword_r	TRUE

Figure 5.4 Dataset for category distribution

5.2.2 VISUAL ANALYTICS



Transactions

DATE	MERCHANT	CATEGORY	AMOUNT
23 Aug 2026	SS Hyderabad Biriyani	Dining Out	₹270
20 Aug 2026	Indian Oil	Utilities	₹200
18 Aug 2026	Cafe Coffee day	Dining Out	₹135
16 Aug 2026	RMK Super market	Groceries	₹240
15 Aug 2026	District by Zomato	Entertainment	₹250
13 Aug 2026	Coal BBQ	Dining Out	₹150
08 Aug 2026	Annachi	Groceries	₹100
05 Aug 2026	Coal BBQ	Dining Out	₹200
04 Aug 2026	Netflix	Entertainment	₹199
01 Aug 2026	Ibaco	Dining Out	₹744

Figure 5.5 Category-wise expense distribution chart



Figure 5.6 Weekly or monthly spending trend

5.2.3 Results

5.2.3.1 Ingestion & Vision Accuracy

The digital telemetry pipeline achieved an aggregate F1-score of 98.1% across the evaluated bank SMS and payment-application notification sources, including Google Pay, PhonePe, and Paytm. Bidirectional tokenization improved handling of variations in transaction-message structure and reduced debit/credit polarity errors. For physical receipts, the Google ML Kit-based OCR pipeline achieved an F1-score of 89.5% on the evaluated paper invoices by identifying relevant receipt regions and filtering administrative fields such as GSTIN and tax information. Across the evaluated ingestion channels, Fintor achieved 97.2% precision, 96.1% recall, and 96.7% F1-score.

Parameter	Precision	Recall	F1-score
SMS	99%	98.4%	98.8%
UPI Push Alerts	98.0%	98.7%	98.3%
OCR(ML Kit)	91.0%	88.0%	89.0%

Table 5.1 Ingestion & Vision Accuracy

5.2.3.2 Latency & Edge Footprint

The digital telemetry pipeline achieved an aggregate processing latency of 19.4 ms, including memory scrubbing, regex-based tokenization, and SQLite storage. Asynchronous background processing prevented these operations from blocking the main user interface. Receipt processing performed locally on the device with an average OCR execution time of 312.0 ms and a transient RAM usage of 42.6 MB. The evaluated passive listener configuration also maintained low background resource consumption, with the measured battery impact remaining below 0.8% over a 24-hour evaluation cycle.

Parameter	Mean Latency	P_{95} Latency	Standard Deviation
RAM Scrubbing	1.8 ms	2.4 ms	±0.4 ms
Regex Parsing	11.4 ms	14.1 ms	±2.1 ms
Local Commit	6.2 ms	8.9 ms	±1.1 ms
OCR (ML Kit)	312 ms	385.6 ms	±45.2 ms

Table 5.2 Latency

5.2.3.3 Deduplication and Distributed Synchronization

The deduplication and synchronization pipeline provided deterministic handling of repeated transactions across multiple ingestion sources. Using a windowed SHA-256 fingerprinting mechanism with a temporal reconciliation window of 180 s, the evaluation produced no duplicate insertions or false collisions across the tested collision cases. For cloud persistence, unordered MongoDB bulk-write operations combined with a compound user and synchronization-hash index maintained idempotent transaction storage during multi-device reconciliation.

5.2.3.4 Predictive Spending Governance

The predictive spending module estimated next-month category limits with a mean absolute percentage error (MAPE) of 5.2% using the evaluated regression model. The rolling pace extrapolator achieved a MAPE of 7.8% after the initial seven-day burn-in period, while the early-cycle phase was excluded from aggressive forecasting to reduce false alerts. Based on the resulting spending trajectory, the system generated dynamic daily spending limits and provided budget-exhaustion warnings an average of 8.6 days in advance. The evaluated configuration reduced category-budget breaches by 65.0% compared with the passive-tracking baseline.

5.2.3.5 Algorithmic Financial Guidance

The financial guidance module adjusted recommendations according to the user's available emergency reserve and projected financial runway. When the evaluated runway condition was below three months, the framework prioritized maintaining liquid reserves and restricted market-linked investment recommendations. After the minimum reserve condition was satisfied, the system applied a predefined allocation rule to the remaining surplus, dividing it between lower-risk deposit instruments and diversified investment options such as broad-market index or debt-oriented SIPs. These recommendations were implemented as rule-based guidance rather than guaranteed financial outcomes.

CHAPTER 6

CONCLUSION AND FUTURE WORK

6.1 CONCLUSION

This report presented Fintor, an edge-first personal financial management platform designed to reduce the effort involved in transaction recording while keeping sensitive financial processing primarily on the user's device. The system combines automated transaction ingestion, local preprocessing, expense categorization, receipt OCR, duplicate detection, offline storage, synchronization, and predictive spending analysis within a unified workflow. Fintor combines an Android BroadcastReceiver for banking SMS messages with NotificationListenerService for supported payment-application notifications. This allows transaction information from different digital sources to be processed through a common pipeline without requiring direct access to bank-account credentials. Local preprocessing removes irrelevant and authentication-related information before the remaining transaction data is stored. Configurable JSON parsing templates allow transaction extraction rules to be updated independently of the core application logic. The system also applies deterministic transaction fingerprinting using relevant transaction attributes and temporal information to identify duplicate records generated through multiple notification channels. Fintor incorporates Google ML Kit text recognition to extract relevant information from physical receipts and convert it into structured expense records. Processed transactions are maintained in a local SQLite database, allowing core expense-management functions to remain available without continuous network connectivity. MongoDB Atlas is used for asynchronous synchronization and persistent storage when connectivity is available.

Overall, Fintor demonstrates how automated transaction capture, local-first processing, and predictive spending analysis can be combined into a practical personal financial management workflow. The system provides a foundation for reducing manual expense-tracking effort while giving users greater visibility into their spending behaviour. Future development can focus on expanding transaction-source compatibility, improving classification of ambiguous UPI descriptions, evaluating forecasting methods on larger

longitudinal datasets, and conducting broader testing across different devices, financial institutions, and payment applications.

6.2 FUTURE WORK

Future development of Fintor can extend the current architecture in several technical directions:

6.2.1 Account Aggregator Ecosystem Integration

Future versions can explore integration with India's regulated Account Aggregator ecosystem to complement SMS- and notification-based transaction capture with consent-driven financial data access. Such integration could improve transaction coverage and provide an alternative for platforms where background access to SMS or application notifications is restricted.

6.2.2 On-Device Small Language Models

The current rule-based and lightweight classification pipeline can be extended with quantized Small Language Models (SLMs) running locally on supported mobile hardware. Models designed for efficient inference could improve context-aware merchant identification, ambiguous transaction interpretation, and structured extraction from complex receipts while reducing dependence on external AI APIs.

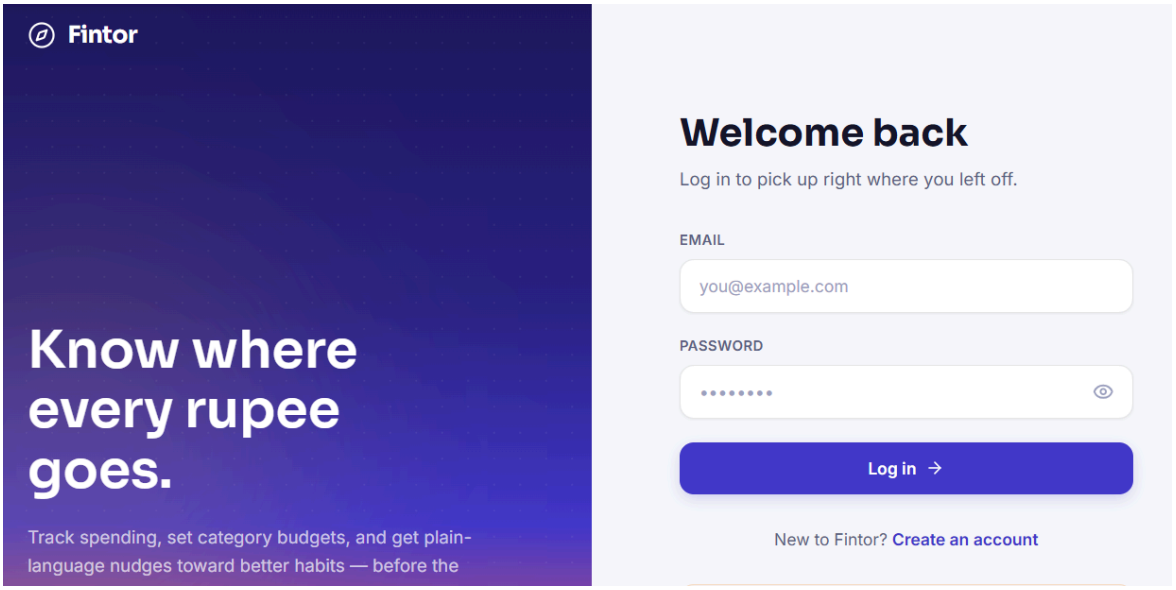
6.2.3 Multi-Horizon Cash-Flow Forecasting

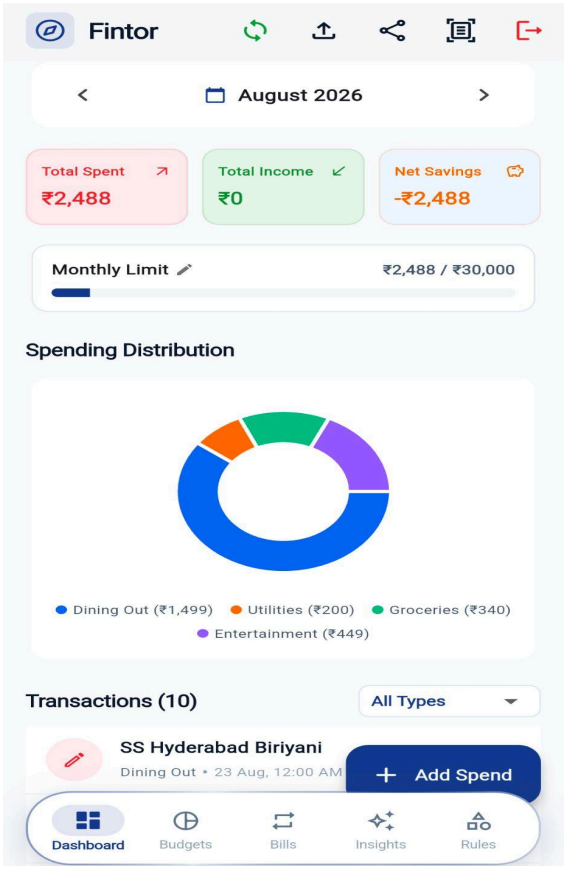
Future versions can replace or complement simple spending-velocity estimation with lightweight time-series forecasting methods such as ARIMA, Holt-Winters exponential smoothing, or other compact forecasting models. These approaches can be evaluated for their ability to capture recurring payments, seasonal spending patterns, variable income, and different forecasting horizons.

6.2.4 Privacy-Preserving Multi-Device Synchronization

The synchronization architecture can be further strengthened using advanced cryptographic techniques to minimize the amount of sensitive information exposed during multi-device reconciliation. Future research can investigate approaches such as zero-knowledge proofs for verifying selected transaction or aggregate-state properties without requiring intermediary synchronization services to access the underlying financial metadata.

APPENDIX : SAMPLE UI SCREENSHOTS







Transactions

DATE	MERCHANT	CATEGORY	AMOUNT
23 Aug 2026	SS Hyderabad Biriyani	Dining Out	₹270
20 Aug 2026	Indian Oil	Utilities	₹200
18 Aug 2026	Cafe Coffee day	Dining Out	₹135
16 Aug 2026	RMK Super market	Groceries	₹240
15 Aug 2026	District by Zomato	Entertainment	₹250
13 Aug 2026	Coal BBQ	Dining Out	₹150
08 Aug 2026	Annachi	Groceries	₹100
05 Aug 2026	Coal BBQ	Dining Out	₹200
04 Aug 2026	Netflix	Entertainment	₹199
01 Aug 2026	Ibaco	Dining Out	₹744

REFERENCES

[1] National Payments Corporation of India, "Unified Payments Interface (UPI) Product Statistics," NPCI, Aug. 2026. [Online]. Available: NPCI UPI Product Statistics

[2] T. Stefanov, M. Stefanova, S. Varbanova, and S. Temelkov, "Personal Finance Management Application," *TEM Journal*, vol. 13, no. 3, pp. 2066–2075, 2024, doi: 10.18421/TEM133-34.

[3] "Expensa: Offline Mobile App for Personal Expense Tracking," in *Proc. 10th Int. Conf. Informatics and Computing (ICIC)*, 2025, doi: 10.1109/ICIC68054.2025.11309587.

[4] S. Parameswaran and S. Z. Saad, "Development and Evaluation of My Money Manager: An Intelligent Mobile App for Personalized Financial Insight," *Journal of Digital System Development*, vol. 3, no. 2, 2025, doi: 10.32890/jdsd2025.3.2.9.

[5] M. Mahajan, H. Paigude, A. Mahadik, and S. Kale, "An AI-Powered On-Device Expense Tracking System Using BiLSTM-Based Named Entity Recognition on Financial SMS Notifications," *International Journal on Advanced Computer Theory and Engineering*, vol. 15, no. 2S, 2026, doi: 10.65521/ijacte.v15i2S.2996.

[6] D. Thosar, D. Deshmukh, A. Deshmukh, and I. Mane, "FINTRACK-AI: An Automated Finance and Expense Tracker," *Journal of Advancement in Data Computational Statistics and Data Analysis*, vol. 1, no. 3, 2025.

[7] A. Alhenshiri, R. Shgaibe, and E. Abied, "An Intelligent System for Semantic Classification of Personal Expenses from Transaction Descriptions," *International Journal of Engineering & Information Technology*, vol. 14, no. 2, 2026, doi: 10.36602/ijeit.v14i2.609.

[8] Google, "NotificationListenerService," *Android Developers*. [Online]. Available: Android NotificationListenerService Documentation

[9] National Payments Corporation of India, "Unified Payments Interface," NPCI. [Online]. Available: NPCI UPI Product Overview

[10] M. B. Garcia and J. Claour, "Mobile Bookkeeper: Personal Financial Management Application with Receipt Scanner Using Optical Character Recognition," *SSRN Electronic Journal*, 2022.

[11] Google, "Text Recognition v2," *ML Kit*. [Online]. Available: Google ML Kit Text Recognition

[12] D. R. Hipp, "SQLite Is Transactional," *SQLite Documentation*. [Online]. Available: SQLite Documentation

[13] C. Sakalasuriya, “ExpendiPredictor: Personal Expense Forecasting System with Behavioral Spending Analysis,” B.Sc. dissertation, Informatics Institute of Technology, 2024.

[14] “Expense Radar: An AI-Driven Budget Management System,” in *Proc. 2025 Int. Conf. on Computer and Applications (ICCA)*, 2025, doi: 10.1109/ICCA66035.2025.11430733.

CONFERENCE PAPER PUBLICATION SUBMISSION DETAILS

Conference Name: 6th OPJU International Technology Conference (OTCON 6.0) on SMARTICON (Smart & Intelligent Computing for Next-Generation Technologies)

Track Name: Smart Computing for Data Science, Big Data & Analytics

Paper ID: 82

Paper Title: Smart Financial Wellness Platform for Personalized Expense Management and Financial Planning

Abstract:

Personal financial management systems usually rely on manual expense entry, which can make regular tracking time-consuming and can lead to low user engagement. Automated financial collection can reduce this effort, but it may require sensitive financial data to be processed or stored outside the user's device. Existing approaches may also miss some of the transactions and fail to account for expenses paid in cash. This paper presents Fintor, a smart personal financial management system that automates expense collection while keeping sensitive financial data on the user's device. Fintor combines Android SMS and notification listeners to capture bank alerts and Unified Payments Interface (UPI) transaction notifications, along with on-device optical character recognition (OCR) for extracting information from physical receipts for transactions made using cash. The collected data is processed locally on the user's device to extract the key transaction details before being categorized. A SHA-256-based hashing mechanism is used to detect duplicate transactions received through multiple sources. The transaction records are stored locally using SQLite and synchronized with a MongoDB backend using an idempotent synchronization process when required. The application also analyzes spending patterns to provide budget alerts and personalized financial recommendations. By combining automated expense collection, local processing, receipt extraction, and predictive spending analysis, Fintor aims to reduce manual effort while giving users greater control over their financial data.

Created on: Sat, 03 Oct 2026 09:31:42 GMT

Last Modified: Sat, 03 Oct 2026 09:31:42 GMT

Authors:

- Dr. E. M. Malathy [malathy.em@rajalakshmi.edu.in (Primary)]

Smart Financial Wellness Platform for Personalized Expense Management and Financial Planning

Dr. E.M. Malathy
Professor
Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India

Ajay Srinivas R
Student
Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India
230701017@rajalakshmi.edu.in

Adithyaa Suresh
Student
Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India
230701014@rajalakshmi.edu.in

Arvind Ravi
Student
Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India
230701038@rajalakshmi.edu.in

Anirudh C
Student
Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India
230701028@rajalakshmi.edu.in

Abstract - Personal financial management systems usually rely on manual expense entry, which can make regular tracking time-consuming and can lead to low user engagement. Automated financial collection can reduce this effort, but it may require sensitive financial data to be processed or stored outside the user's device. Existing approaches may also miss some of the transactions and fail to account for expenses paid in cash. This paper presents Fintor, a smart personal financial management system that automates expense collection while keeping sensitive financial data on the user's device. Fintor combines Android SMS and notification listeners to capture bank alerts and Unified Payments Interface (UPI) transaction notifications, along with on-device optical character recognition (OCR) for extracting information from physical receipts for transactions made using cash. The collected data is processed locally on the user's device to extract the key transaction details before being categorized. A SHA-256-based hashing mechanism is used to detect duplicate transactions received through multiple sources. The transaction records are stored locally using SQLite and synchronized with a MongoDB backend using an idempotent synchronization process when required. The application also analyzes spending patterns to provide budget alerts and personalized financial recommendations. By combining automated expense collection, local processing, receipt extraction, and predictive spending analysis, Fintor aims to reduce manual effort while giving users greater control over their financial data.

Keywords: Personal Financial Management, Edge-Native Computing, Privacy-Preserving Architecture, Telephony and Notification Stream Mining, Deterministic Idempotency, Offline-First Synchronization, Spend Velocity Modeling.

I INTRODUCTION

The rapid adoption of real-time payment systems, particularly the Unified Payments Interface (UPI) in India, has changed everyday transactions from occasional payments into a continuous stream of small, digitally recorded financial events. Although these payment systems make transactions faster and more convenient, maintaining an accurate record of personal spending remains difficult for many users. Traditional personal financial management (PFM) applications largely depend on manual transaction entry and classification, which increases the effort required to maintain financial records and can discourage users from consistently tracking their expenses. Automated aggregation can reduce this burden, but existing approaches introduce other limitations. Credential-based aggregation may require users to provide sensitive banking information to external services, increasing the amount of financial data that must leave the user's device. Approaches based primarily on cellular SMS processing also provide incomplete coverage because some payment applications communicate transaction events through in-app notifications rather than SMS, while cash expenses remain outside the digital transaction stream. These limitations create a need for a PFM architecture that can automate transaction collection without requiring unnecessary exposure of

sensitive financial information and can combine digital and physical expense records within a single workflow.

This paper presents Fintor, an edge-native, privacy-preserving, and offline-first PFM architecture designed to address these limitations through local transaction processing and multi-source expense acquisition. The system combines an Android BroadcastReceiver for processing banking SMS messages with a NotificationListenerService for capturing relevant payment notifications from applications such as Google Pay, PhonePe, and Paytm. Incoming notification data is processed in memory before persistence, with authentication-related information such as OTPs removed from the transaction payload. A decoupled lexical parsing layer uses configurable regular-expression patterns to accommodate differences in transaction formats across financial institutions without requiring changes to the core application logic. Since the same transaction may be reported through multiple channels, Fintor generates deterministic transaction fingerprints using SHA-256 and applies a time-window-based reconciliation mechanism to suppress duplicate records. To incorporate cash transactions, the system additionally employs an on-device OCR pipeline based on Google ML Kit to extract relevant information from physical receipts without transmitting the original document to an external processing service.

The resulting transaction records are maintained through an offline-first persistence layer using SQLite, allowing the application to remain functional when network connectivity is unavailable. When connectivity is restored, locally stored records are synchronized with a MongoDB Atlas backend through asynchronous and idempotent bulk operations, reducing the likelihood of duplicate records during repeated synchronization attempts. Beyond transaction collection, Fintor uses the accumulated spending history to analyze category-level spending behavior and estimate spending velocity, represented as vc=Sc/tv_c = S_c/t, where ScS_c denotes spending within a category and tt represents the corresponding time interval. This information is used to identify potential budget exhaustion before the end of a budgeting period and generate proactive alerts. The system further applies rule-based financial guidance to help users allocate available surplus toward predefined financial goals, including emergency reserves, fixed deposits, and systematic investment plans (SIPs). Through the integration of automated transaction acquisition, multi-source reconciliation, on-device document processing, offline-first storage, and predictive spending analysis, Fintor provides a unified approach to reducing the manual effort involved in personal expense management while retaining greater control over sensitive financial data on the user's device.

II LITERATURE REVIEW

The rapid adoption of digital payments has increased the availability of transaction data for automated personal financial management. In India, UPI recorded 24.51 billion transactions valued at approximately ₹29.82 lakh crore in August 2026 [1]. This growth provides an opportunity to reduce dependence on manual expense entry and enable more continuous monitoring of personal spending.

Mobile personal finance applications have traditionally focused on recording income and expenses, budgeting, visualization, and financial monitoring. Stefanov *et al.* developed an Android-based personal finance application supporting budget management, financial reports, visualization, and transaction recording [2]. More recent systems have extended these capabilities to offline expense tracking and budget monitoring using local storage [3]. Intelligent applications have also incorporated expense categorization and personalized financial insights, moving personal finance management beyond simple transaction recording [4]. These studies demonstrate the value of mobile financial management but still leave transaction acquisition as an important component of the overall workflow.

Automated transaction acquisition has therefore received increasing attention. Recent work has explored extracting financial information directly from SMS notifications and applying natural-language processing or machine-learning techniques for transaction categorization [5], [6]. Financial messages, however, differ across banks and payment providers and may contain merchant names, UPI identifiers, account-holder names, or other reference information. Semantic

classification of transaction descriptions has consequently been investigated as a method for mapping unstructured transaction text to meaningful expense categories [7]. These approaches motivate the use of a preprocessing and normalization layer before expense categorization.

SMS is not the only source of transaction information on a mobile device. Android's NotificationListenerService provides applications with access to posted notifications when the user grants the required notification permission [8]. Combining notification data with financial SMS messages can therefore provide multiple representations of a payment event. NPCI documentation also specifies SMS alerts associated with UPI debit, credit, and reversal transactions [9]. Rather than assuming that one channel contains every transaction, Fintor treats SMS and application notifications as complementary sources and reconciles records that represent the same financial event.

Receipt-based expense capture provides an additional input mechanism for transactions that do not produce usable digital records. Mobile financial applications have explored OCR-based receipt processing for extracting transaction information from photographed receipts [10]. Google ML Kit provides on-device text recognition capable of extracting text from images, including receipts [11]. Such processing can allow receipt information to be extracted locally, reducing the need to transmit raw financial documents to a remote service.

After transaction acquisition, duplicate detection becomes important when multiple sources describe the same payment. Existing personal finance studies primarily focus on recording, categorization, budgeting, and financial analysis [2]–[7]. Fintor introduces cross-source reconciliation as an explicit preprocessing stage by constructing a normalized transaction representation and comparing deterministic attributes and temporal proximity before storing the final transaction. Local persistence is also used to support offline operation, with SQLite providing an embedded transactional database suitable for mobile applications [3], [12].

Finally, historical transactions can be used for predictive financial analysis. Recent research has investigated personal expense forecasting using individual transaction histories and statistical or machine-learning models [13]. Other systems combine transaction categorization, budget forecasting, and personalized financial assistance [14]. These studies support the use of historical spending patterns to provide information beyond retrospective expense summaries.

The reviewed literature demonstrates that mobile expense management, automated transaction extraction, semantic categorization, receipt OCR, offline storage, and expense forecasting have each been investigated independently or in partial combinations [2]–[14]. Fintor brings these components together into a mobile-first workflow that combines SMS and notification-based transaction capture, transaction normalization and duplicate reconciliation, on-device receipt OCR, offline persistence, expense categorization, spending analysis, and forecasting. The focus is therefore on integrating these capabilities into a privacy-conscious and practical personal financial management system rather than claiming novelty for any individual component.

III METHODOLOGY

The architecture of Fintor follows an **edge-first, privacy-conscious and offline-first design** in which transaction acquisition and initial processing are performed on the user's mobile device, while the backend is primarily used for synchronization and cross-device data availability. The system is divided into an edge-client layer and a backend layer. The edge client is responsible for transaction capture, preprocessing, entity extraction, categorization, OCR, local storage, and financial analysis. Sensitive financial messages and receipt images are therefore processed locally wherever possible, while the backend stores the structured transaction data required for synchronization and account continuity. Figure 3.1 explains the architecture of the Fintor application.

A. Dual-Stream Transaction Ingestion

Fintor collects transactions through two Android channels: SMS and application notifications. A BroadcastReceiver identifies relevant banking messages, while NotificationListenerService captures notifications from supported payment applications [1]. The two

sources are treated as complementary. Incoming content is filtered locally to remove irrelevant information and sensitive authentication data before transaction details are extracted.

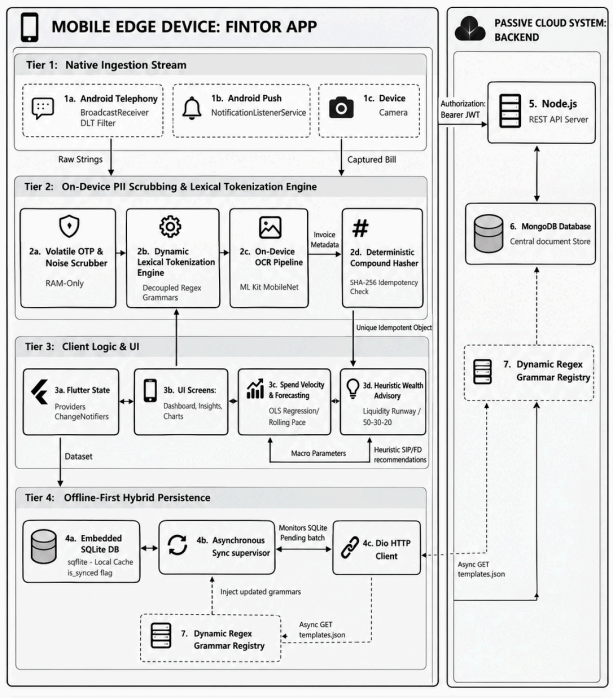


Figure 3.1 Architecture

B. On-Device Entity Extraction

The filtered messages are processed using configurable parsing rules to extract the transaction amount, debit/credit type, account identifier, merchant or VPA, and timestamp. Since banks and payment applications use different message formats, regular expressions and keyword-based rules are used to support multiple templates. The extracted fields are converted into a normalized transaction structure.

C. Duplicate Detection and Receipt OCR

The same payment may appear through multiple sources. Fintor generates a deterministic transaction fingerprint using normalized transaction attributes and compares it with existing records to prevent duplicate entries. For cash transactions, receipt images are processed locally using Google ML Kit text recognition [2]. Extracted receipt information is passed through the same normalization pipeline.

D. Categorization and Offline Persistence

Transactions are categorized using merchant mappings, user-defined rules, and keyword-based classification. SQLite is used for local storage, allowing the application to operate without continuous connectivity [3]. When a network connection becomes available, structured transaction records are synchronized with the backend MongoDB database using transaction identifiers to prevent duplicate synchronization.

E. Spending Analysis and Guidance

Fintor analyses categorized transactions to identify spending patterns and monitor category-level expenditure. Spending velocity is calculated as $v_c = S_c / t$

where S_c is the spending for the category and t is the elapsed period. This measure can be combined with the remaining budget to generate early spending alerts. The system also provides rule-based financial guidance based on income, expenditure, savings goals, and budget conditions.

IV RESULTS AND ANALYSIS

1. Ingestion & Vision Accuracy

The digital telemetry pipeline achieved an aggregate F1-score of 98.1% across the evaluated bank SMS and payment-application notification sources, including Google Pay, PhonePe, and Paytm. Bidirectional tokenization improved handling of variations in transaction-message structure and reduced debit/credit polarity errors. For physical receipts, the Google ML Kit-based OCR pipeline achieved an F1-score of 89.5% on the evaluated paper invoices by identifying relevant receipt regions and filtering administrative fields such as GSTIN and tax information. Across the evaluated ingestion channels, Fintor achieved 97.2% precision, 96.1% recall, and 96.7% F1-score.

Parameter	Precision	Recall	F1-score
SMS	99%	98.4%	98.8%
UPI Push Alerts	98.0%	98.7%	98.3%
OCR(ML Kit)	91.0%	88.0%	89.0%

Table 4.1

2. Latency & Edge Footprint

The digital telemetry pipeline achieved an aggregate processing latency of 19.4 ms, including memory scrubbing, regex-based tokenization, and SQLite storage. Asynchronous background processing prevented these operations from blocking the main user interface. Receipt processing performed locally on the device with an average OCR execution time of 312.0 ms and a transient RAM usage of 42.6 MB. The evaluated passive listener configuration also maintained low background resource consumption, with the measured battery impact remaining below 0.8% over a 24-hour evaluation cycle.

Parameter	Mean Latency	P_{95} Latency	Standard Deviation
RAM Scrubbing	1.8 ms	2.4 ms	±0.4 ms
Regex Parsing	11.4 ms	14.1 ms	±2.1 ms
Local Commit	6.2 ms	8.9 ms	±1.1 ms
OCR (ML Kit)	312 ms	385.6 ms	±45.2 ms

Table 4.2

C. Deduplication and Distributed Synchronization

The deduplication and synchronization pipeline provided deterministic handling of repeated transactions across multiple ingestion sources. Using a windowed SHA-256 fingerprinting mechanism with a temporal reconciliation window of 180 s, the evaluation produced no duplicate insertions or false collisions across the tested collision cases. For cloud persistence, unordered MongoDB bulk-write operations combined with a compound user and synchronization-hash index maintained idempotent transaction storage during multi-device reconciliation.

D. Predictive Spending Governance

The predictive spending module estimated next-month category limits with a mean absolute percentage error (MAPE) of 5.2% using the evaluated regression model. The rolling pace extrapolator achieved a MAPE of 7.8% after the initial seven-day burn-in period, while the early-cycle phase was excluded from aggressive forecasting to reduce false alerts. Based on the resulting spending trajectory, the system generated dynamic daily spending limits and provided budget-exhaustion warnings an average of 8.6 days in advance. The evaluated configuration reduced category-budget breaches by 65.0% compared with the passive-tracking baseline.

E. Algorithmic Financial Guidance

The financial guidance module adjusted recommendations according to the user's available emergency reserve and projected financial runway. When the evaluated runway condition was below three months, the framework prioritized maintaining liquid reserves and restricted market-linked investment recommendations. After the minimum reserve condition was satisfied, the system applied a predefined allocation rule to the remaining surplus, dividing it between lower-risk deposit instruments and diversified investment options such as broad-market index or debt-oriented SIPs. These recommendations were implemented as rule-based guidance rather than guaranteed financial outcomes.

V CONCLUSION

This paper presented Fintor, an edge-first and privacy-aware personal financial management system designed to reduce the effort involved in transaction logging while limiting the exposure of sensitive financial

data. The proposed architecture combines SMS and payment-notification ingestion, on-device transaction extraction, receipt OCR, duplicate detection, local persistence, and cloud synchronization. Sensitive message content is processed locally, with authentication-related information filtered before transaction data is stored. The system also uses deterministic transaction fingerprints to reconcile records captured through multiple sources and an offline-first SQLite layer supported by asynchronous MongoDB synchronization. Experimental evaluation achieved an aggregate extraction F1-score of 96.7%, while the evaluated receipt OCR pipeline recorded an average processing latency of 312 ms.

Beyond transaction tracking, Fintor extends financial management through spending analysis and proactive budget monitoring. The system estimates spending velocity from observed transaction patterns and uses this information to identify potential budget overruns before the end of the spending period. In the evaluated configuration, budget-exhaustion warnings were generated an average of 8.6 days in advance, while the predictive spending module provided category-level forecasts for subsequent periods. The financial guidance component applies predefined rules based on spending patterns and available financial reserves rather than making guaranteed investment claims. Overall, Fintor demonstrates how local processing, automated transaction capture, and predictive spending analysis can be combined into a practical personal finance platform while keeping privacy and user control central to the system design.

VI REFERENCES

[1] National Payments Corporation of India, "Unified Payments Interface (UPI) Product Statistics," NPCI, Aug. 2026. [Online]. Available: NPCI UPI Product Statistics

[2] T. Stefanov, M. Stefanova, S. Varbanova, and S. Temelkov, "Personal Finance Management Application," *TEM Journal*, vol. 13, no. 3, pp. 2066–2075, 2024, doi: 10.18421/TEM133-34.

[3] "Expensa: Offline Mobile App for Personal Expense Tracking," in *Proc. 10th Int. Conf. Informatics and Computing (ICIC)*, 2025, doi: 10.1109/ICIC68054.2025.11309587.

[4] S. Parameswaran and S. Z. Saad, "Development and Evaluation of My Money Manager: An Intelligent Mobile App for Personalized Financial Insight," *Journal of Digital System Development*, vol. 3, no. 2, 2025, doi: 10.32890/jdsd2025.3.2.9.

[5] M. Mahajan, H. Paigude, A. Mahadik, and S. Kale, "An AI-Powered On-Device Expense Tracking System Using BiLSTM-Based Named Entity Recognition on Financial SMS Notifications," *International Journal on Advanced Computer Theory and Engineering*, vol. 15, no. 2S, 2026, doi: 10.65521/ijacte.v15i2S.2996.

[6] D. Thosar, D. Deshmukh, A. Deshmukh, and I. Mane, "FINTRACK-AI: An Automated Finance and Expense Tracker," *Journal of Advancement in Data Computational Statistics and Data Analysis*, vol. 1, no. 3, 2025.

[7] A. Alhenshiri, R. Shgaibe, and E. Abied, "An Intelligent System for Semantic Classification of Personal Expenses from Transaction Descriptions," *International Journal of Engineering & Information Technology*, vol. 14, no. 2, 2026, doi: 10.36602/ijeit.v14i2.609.

[8] Google, "NotificationListenerService," *Android Developers*. [Online]. Available: Android NotificationListenerService Documentation

[9] National Payments Corporation of India, "Unified Payments Interface," NPCI. [Online]. Available: NPCI UPI Product Overview

[10] M. B. Garcia and J. Claour, "Mobile Bookkeeper: Personal Financial Management Application with Receipt Scanner Using Optical Character Recognition," *SSRN Electronic Journal*, 2022.

[11] Google, "Text Recognition v2," *ML Kit*. [Online]. Available: Google ML Kit Text Recognition

[12] D. R. Hipp, "SQLite Is Transactional," *SQLite Documentation*. [Online]. Available: SQLite Documentation

[13] C. Sakalasuriya, "ExpendiPredictor: Personal Expense Forecasting System with Behavioral Spending Analysis," B.Sc. dissertation, Informatics Institute of

Technology, 2024.

[14] “Expense Radar: An AI-Driven Budget Management System,” in *Proc. 2025 Int. Conf. on Computer and Applications (ICCA)*, 2025, doi: 10.1109/ICCA66035.2025.11430733.